TECHNICAL REPORT

DRAFT v0.1 · PILOT

Continuous Architecture Assurance

Calibrating AI reviewers as instruments — a framework, an experiment, and what five attempts to break it taught me.

> *CAA does not review your architecture. It is the mechanism for measuring whether the things that review your architecture deserve to be trusted.*

Rudson Carvalho · Independent Researcher

Companion to the CAA position, empirical, and meta papers

Executive summary

Read this page if you read nothing else.

Software teams are starting to let AI review their architecture and code. The reflex is to trust it: the model sounds competent, so its verdict is accepted. This report is about the question nobody asks before doing that — **how do you know the AI reviewer is any good?**

The market is full of "rulers" for judging software quality: well-architected frameworks, maturity models, checklists, and now AI reviewers. What nobody sells is the thing that tells you whether a ruler measures what it claims. **Continuous Architecture Assurance (CAA) is that thing — the mechanism that calibrates the rulers.** It treats an AI reviewer not as an oracle to be trusted but as an *instrument to be calibrated*, the way a metrology lab calibrates a measuring device against a known standard. The standard, here, is real incidents: does the reviewer catch the risk that actually caused a real outage, and does it avoid crying wolf on a healthy system?

I built CAA as a framework, then spent five experimental attempts trying to falsify its central claim instead of confirm it. The honest result:

- **It works on one axis, demonstrably and repeatably.** A reviewer working under a strict contract is measurably better than a naive one, and both are measurably better than deliberately broken reviewers — and this replicated across two independent sets of real incidents.
- **It could not be shown to work on a second axis**, and I explain exactly why (a property of how language models read text, not a flaw I can patch).
- **A third capability remains untested**, because the reviewer failure mode most worth catching turned out to be surprisingly hard to even simulate.
- **And the most valuable finding arrived sideways:** the automated quality gate I built to police the experiment failed to catch a serious defect in *all five* iterations. Every time, only a human reading the raw data caught it. That is not an embarrassing footnote — it is the strongest possible evidence for the one thing CAA insists on: the party that assures cannot be the party that produces.

If you are deciding whether something like CAA is worth adopting, the short version is: **yes, for AI-assisted review in domains where being confidently wrong is expensive — and the reason to trust it is precisely that I am telling you where it does not work.**

The rest of this document explains the problem, the idea, how I measured it, the corpus, the full five-attempt saga with its dead ends, the verdict, and how you would actually start using this.

Claims and evidence at a glance

The honest scorecard, before the narrative. Every number in this report is reconstructed from raw per-call records, independently of the experiment harness, with run provenance verified as real.

Claim	Status	Evidence	Key limitation
Contract-bound review reduces false positives vs. naive review	**Demonstrated**	Two independent corpora, real model calls (FP 0.05 vs 0.25; 0.10 vs 0.55)	Controlled *textual* incident pairs, not live systems

Claim	Status	Evidence	Key limitation
The paired recall/FP metric catches degenerate reviewers	**Demonstrated**	Over-flagger caught by FP; coverage-gap reviewer caught by recall	Degenerate controls are constructed, not naturally occurring
Recall discriminates among *plausible* reviewers	**Not demonstrated**	Naive recall saturated (1.00) on both corpora	Textual cases appear too easy for a competent model to miss
The metric catches a confident-misattribution reviewer	**Untested**	The control could not be constructed across three attempts	May require a real, not synthetic, misreasoning source
A closed-rule automated assurance gate is sufficient on its own	**Refuted / weakened**	The gate accepted a defect in all five runs; human audit caught each	Independent human audit remains required

The two demonstrated rows are the core result. The two "not demonstrated / untested" rows are honest gaps, each with a stated reason. The last row is the recursive finding — and, I will argue, the most important one.

Scope boundary, stated plainly: this work does *not* establish that CAA detects architectural risk directly from production systems, codebases, telemetry, or live design artifacts. It establishes that a contract-bound reviewer can be calibrated against controlled textual incident pairs, and that doing so exposed concrete failure modes in both the reviewers and the assurance gate itself.

Part 1 — The problem worth solving

1.1 The trust reflex

When an AI reviewer says "this architecture has a retry-storm risk," what happens next? In most teams, one of two things: the team trusts it and acts, or the team ignores it as noise. Both are failures of the same kind — neither is based on any *measurement* of whether this reviewer, on this kind of problem, is reliable.

We would never accept this from a physical instrument. A thermometer that has never been calibrated is not a thermometer; it is an opinion with a number on it. Yet we deploy AI reviewers — which are far less stable than thermometers — and accept their readings on faith.

1.2 Why "more reviewers" is not the answer

The common response to low trust is to add reviewers: one agent for security, one for data, one for capacity, each emitting pages of feedback. In practice this degrades into what I call **checklist theater** — a volume of plausible-sounding output that nobody arbitrates, frequently built on failure scenarios the model simply invented, accepted because the reviewers are *assumed* competent. Ten unreliable instruments do not add up to one reliable measurement. They add up to more noise with a quorum.

1.3 The real question

The question is not "how do we get more AI review?" It is: **when an AI participates in judging whether a system is sound, what assures the judge?** Reliability engineering itself — service-level objectives, idempotency, observability, safe rollback — is not in doubt. What is new and unanswered is the assurance of the *reviewer*. That is the gap CAA targets.

Part 2 — The idea: reviewers as calibrated instruments

2.1 The metrology reframing

The move at the heart of CAA is to stop treating AI reviewers as oracles and start treating them as **instruments that must be calibrated**. This single reframing imports a century of mature practice from metrology:

Metrology	CAA
The instrument	An AI reviewer (or any "ruler" — well-architected, maturity model, checklist)
Calibration against a reference standard	Measuring the reviewer against a corpus of **real incidents**
Measurement uncertainty	The residual risk we *knowingly* accept — stated, not hidden
Metrological traceability	Every finding cites a traceable origin
Chain of custody	The verdict contract plus an audit trail

The consequence that makes this powerful for a business: CAA is **ruler-agnostic**. It does not compete with your well-architected framework or your internal checklist or your AI reviewers. Any of them plugs in as an instrument — *provided it agrees to be calibrated and to emit its findings under contract.* The party that calibrates must be independent of the parties that build rulers, which is exactly why this space was empty when I looked: a ruler vendor has no incentive to build the mechanism that can fail its own ruler.

2.2 What CAA actually produces

CAA does not produce "the system will not fail" — that is a fantasy, and any framework that promises it is selling something. It produces **justified confidence with known coverage**: a statement of what was claimed, the evidence for each claim, and the residual risk that was accepted because it was neither claimed nor tested. The unit of work is not a feature; it is a *decision* and the *risk* attached to it.

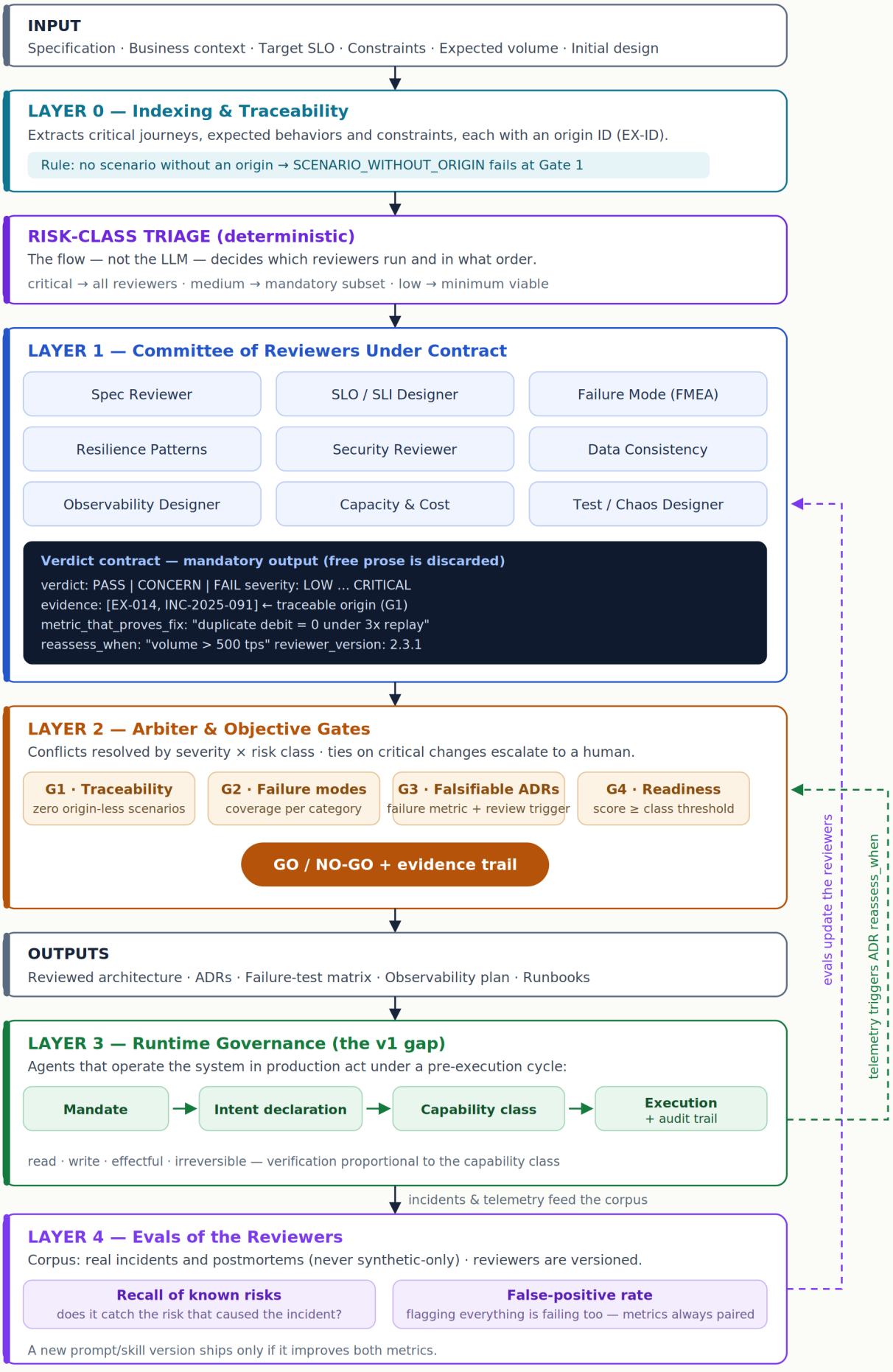
2.3 The two planes, and the one rule that cannot be broken

CAA separates a **production plane** — whatever produces code, decisions, and deployments, whether an AI pipeline, a software factory, or a human team — from an **assurance plane**, which consumes claims and evidence and emits auditable confidence. The interface between them is a structured **verdict contract**.

The one non-negotiable rule is **independence**: the party that assures cannot be the party that produces. A production pipeline's own internal reviewers and quality gates are part of *production*; CAA audits them too. This is the same logic as internal controls versus independent audit in a bank — the first does not remove the need for the second, and regulators require both. Hold onto this rule. By the end of the saga, the experiment itself will have proven, five times over, why it is the load-bearing wall of the whole design.

AI-Assisted Reliability Pipeline — v2

Reviewers under contract · Objective arbitration · Runtime governance · Continuous evals



Feedback loops close the cycle: production feeds back into the decisions (ADRs) and the reviewers' own quality (evals).

AI-Assisted Reliability Pipeline — v2 · reviewers as components: with a contract, gates, telemetry and their own failure criteria.

The CAA assurance pipeline, layer by layer.

Part 3 — How the pipeline works

The assurance plane is organized in layers. I will keep this at the level a decision-maker needs; the full specification is in the position paper and the JSON schema.

Layer 0 — Traceability. Before any reviewer runs, every critical journey and constraint gets an origin tag pointing to the exact source or to a real incident. One rule governs everything downstream: *no scenario without an origin.* A reviewer that invents a risk with no traceable source is auto-rejected. This kills the "plausible noise" that makes checklist theater feel rigorous while being unauditable.

Risk-class triage. A deterministic step — not the AI — decides which reviewers run and in what order, by the risk class of the change. A payment route gets everything; a formatting helper gets the minimum. This bounds cost and removes the chance of the model silently skipping a required check.

Layer 1 — Reviewers under contract. The reviewers emit *only* a structured verdict; free prose is discarded. Three fields carry the weight: the **evidence** (traceable origin, or the finding does not count), the **metric that proves the fix** (no metric, no closed finding — this is what stops decorative architecture), and the **condition to reassess** (which turns each decision into a falsifiable hypothesis).

Layer 2 — Arbiter and gates. Conflicts between reviewers are resolved by an objective rule, not ad hoc; ties on critical changes escalate to a human. Four gates decide advancement, producing a **GO / NO-GO with a full evidence trail**.

Layer 3 — Runtime governance. Because increasingly the AI does not just design the system but operates it, production agents act under a pre-execution cycle: a scoped mandate, a declared intention before acting, a capability classification (read / write / effectful / irreversible) with verification proportional to the risk, and an audit trail.

Layer 4 — Evals of the reviewers. The reviewers are components, so they get their own quality contract — measured against real incidents, with two metrics that must always travel as a pair. This layer is where the metrology lives, and it is what the entire experiment in Part 5 stresses.

Part 4 — How I measured it, and the corpus

4.1 The two-number measurement

A reviewer is calibrated on two numbers that must always be reported together:

- **Recall** — of the real failures, how many did the reviewer catch? A reviewer that misses real risks is useless.
- **False-positive rate** — on healthy systems, how often did the reviewer cry wolf? A reviewer that flags everything is *also* useless; it just moves the cost to the human who has to filter the noise.

The pairing is the whole point. Either number alone is gameable. A reviewer that flags everything scores perfect recall — and is worthless. A reviewer that approves everything scores zero false positives — and is worthless. Only the *pair* exposes both. This is the claim the experiment sets out to break.

4.2 The corpus: matched pairs from real incidents

I could not calibrate against opinions, so I calibrated against reality: public postmortems of real outages from AWS, GitHub, Cloudflare, CircleCI, Mozilla, Twilio, Val Town and others. For each incident I wrote a **matched pair**:

- a **failure case** — the pre-incident architecture, with the real causal mechanism present but *not named*;
- a **healthy twin** — the identical architecture with the mitigation in place.

The pair is the experimental control. Because the twins differ only by the mitigation, any difference in the reviewer's verdict between them isolates its ability to discriminate the *risk* from everything else — writing style, domain, length. Cases were blinded against memorization (fictional names, swapped domains), and every failure case passed a "blind reader" test: no sentence may state the conclusion, and no taxonomy label may appear in the text. (Getting this right took three rewrites — see the saga.) If you want to see exactly which real incidents became which cases, the 12-category taxonomy, and how each pair was built and blinded, the corpus datasheet (`docs/corpus-datasheet.md`) documents all of it with per-pair provenance.

4.3 Pre-registration: the defense against fooling myself

Before every run, I committed the decision criteria in writing — what would count as success, what as failure, what as inconclusive. This is the single most important methodological choice in the whole program. It makes it impossible to look at the numbers and then decide what they meant. The verdict logic could not be retrofitted to the result I was hoping for. If you take one practice from this document into your own work, take this one.

Part 5 — The saga: five attempts to break it

This is the part most reports would hide. I am putting it at the center, because the dead ends are the most valuable content here — each one is a live demonstration of how easily an AI-review process can fool you, and what it takes to catch it. If you are considering letting AI review your systems, read this part as a series of near-misses you would almost certainly have accepted.

Iteration 1 — the control that refused to be bad

I built two real reviewers (a naive one and a contract-bound one) and two deliberately broken "degenerate" controls, including an *over-approver* meant to be a bad reviewer that approves too much. The pre-registered rule said: if a degenerate control sneaks through, the metric has failed.

What happened: the over-approver *refused to be bad.* Instructed to be lax, the model kept reasoning well anyway and caught almost everything. The automated gate, applying the rule literally, declared the whole hypothesis **falsified**. But that was wrong — the control had simply failed to become degenerate, which by the rules means *inconclusive for that arm*, not failure of the whole idea. The gate stated "falsified" with confidence; reading the raw numbers showed it should have said "inconclusive."

Lesson: you cannot create a bad reviewer just by *telling* a capable model to be bad. And the automated verdict was already overstating. First crack in the gate.

Iteration 2 — the corroboration that was true but too broad

I replaced the failed control with a *structurally* broken one — a reviewer blinded to most risk categories by construction, not by persuasion. This time the controls behaved, and the metric cleanly separated the good reviewers from the broken ones. The gate said **corroborated**.

It was right — but reading the raw data showed it was *too broad*. All the discrimination came from one of the two numbers (false positives); the other (recall) was saturated and did no work. "The paired metric works"

overstated what the data licensed. The honest statement was "the metric works on one axis; the other was not tested." This is the iteration whose numbers later anchored the replication claim — verified twice, from raw records.

Lesson: a correct verdict can still be an overclaim. The gate counts; it does not scope.

Iteration 3.0 — the control that collapsed into another control

I added the reviewer I most wanted to test: one that *looks rigorous but attributes the wrong cause* — the dangerous, realistic failure mode. I specified it by instruction ("be confidently wrong"). It collapsed into flagging *everything* across nearly all categories — behaviorally identical to the over-flagger I already had. The gate passed it on a technicality. Reading the per-case findings showed the collapse.

Lesson, repeated: instruction does not produce subtle degeneracy. And the gate, again, passed a malformed control.

Iteration 3.1, first run — the result that was never real

I rebuilt the misattribution control structurally and reran. The gate said **corroborated**, with a beautiful recall number suggesting the corpus had finally been made hard enough.

It was entirely **dry-run mock data.** A hardcoded value in the simulation, presented through the same files as a real run. There was no model call at all. The gate had no check for "is this real?" I caught it only by noticing every log row was flagged `dry_run = true`. I had, one step earlier, started analyzing those mock numbers as if they were real — so the human arm was not infallible either; it was *recoverable*, which the gate was not.

Lesson: the most dangerous result is the one that confirms what you hoped. And "is this data even real?" is a check no automated gate had — but a human reading raw records does.

Iteration 3.1, real run — the verdict, and the control that still could not be built

The real run, finally clean. Both real reviewers and the structural controls behaved; the contract reviewer beat the naive one on false positives by a wide margin, replicating iteration 2 on a fully independent corpus. But two things did not resolve:

- **Recall saturated again.** Even after three corpus rewrites for subtlety, the naive reviewer caught every real failure. A language model reading an architecture description with the cause present in the mechanisms *finds it*, however obliquely it is written. This is not a corpus defect — it replicated across two independent corpora, which makes it a finding: **textual cases cannot stress the recall axis for a competent model reader.**
- **The misattribution control still could not be built.** The structural version, by excluding the correct category from its outputs, made a certain kind of error mechanically impossible — so its score was an artifact of construction, not a measurement. Across three attempts (instruction, blind displacement, affinity displacement) a reviewer that *genuinely reasons wrongly while appearing rigorous* could not be synthesized.

Lesson: the failure mode most worth catching may be an emergent property of flawed reasoning, not something you can fake. That itself is a result.

The pattern across all five attempts

A note on counting, because "five iterations" can sound like rhetorical accounting. These were not five symmetric versions of one experiment. They were **five distinct runs, each a real opportunity for the automated gate to catch a defect**: two initial iterations on independent corpora, one run whose control collapsed, one run that turned out to be contaminated mock data, and one final clean real run. What matters is

not the count but the *heterogeneity* — the gate failed in a different way each time, which is precisely what a single recurring bug would not do.

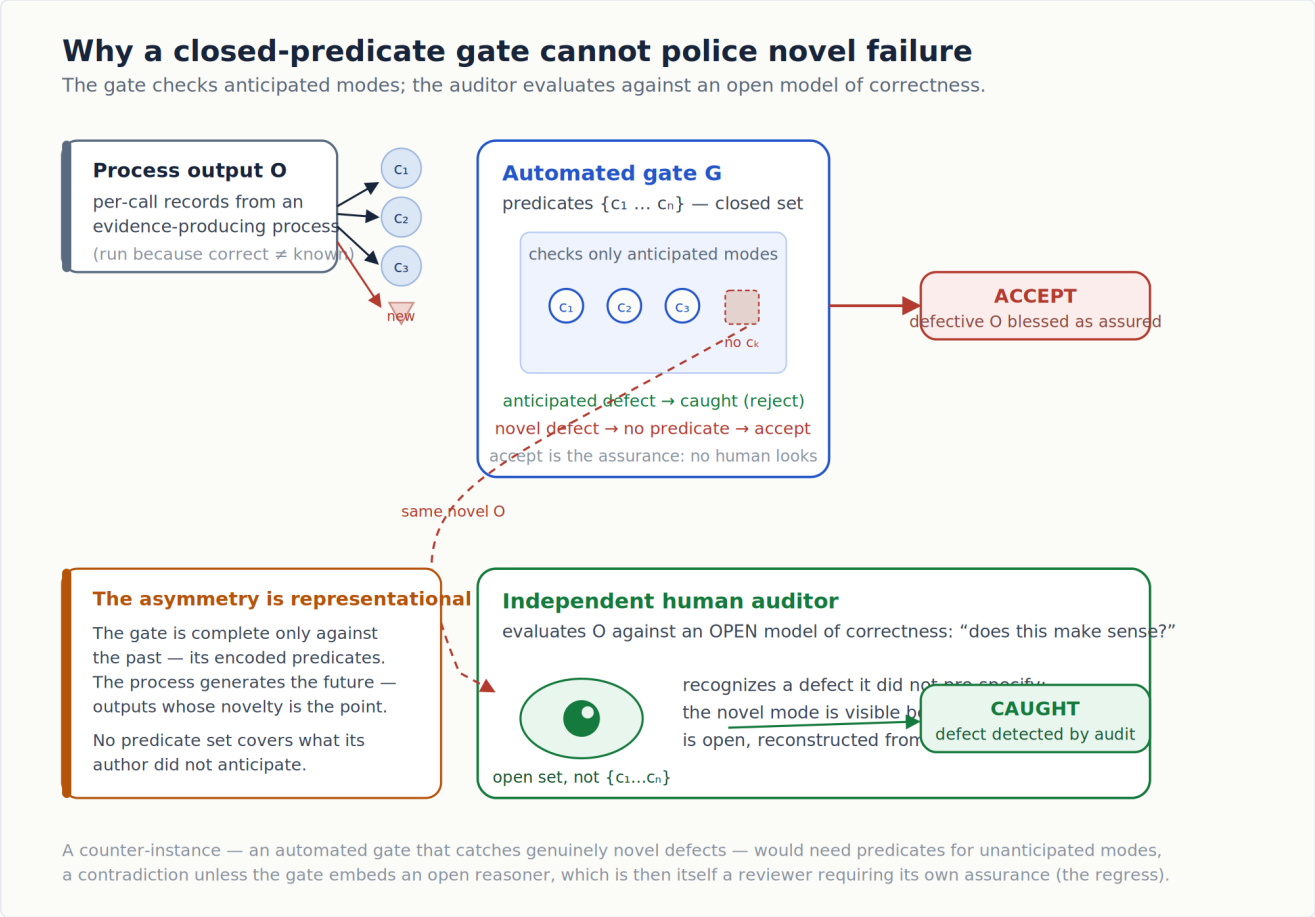
Look back at the five gate verdicts: *falsified, corroborated, inconclusive, corroborated, inconclusive.* In every attempt, the automated gate accepted a structurally defective result — an overstated negative, an over-broad positive, a collapsed control, mock data as real, a structurally impossible score marked valid — and in every attempt, only a human reading the raw records caught it. The gate caught what it was built to catch and passed every new kind of problem.

The numbers

The two clean real runs, reconstructed from raw records (FP = false-positive rate on healthy twins; recall = correct-cause rate on failure cases; n = 20 matched pairs = 40 cases per reviewer per corpus):

Reviewer	Corpus A — recall	Corpus A — FP	Corpus B — recall	Corpus B — FP	Role
Contract (R2)	1.00	**0.05**	1.00	**0.10**	real reviewer
Naive (R1)	1.00	0.25	1.00	0.55	real reviewer (baseline)
Coverage-gap (R3)	0.35	0.00	0.30	0.05	degenerate control
Over-flagger (R4)	1.00	1.00	1.00	1.00	degenerate control
Misattribution (R5)	—	—	0.00	0.00	VOID (uninterpretable)

Read the table for the *shape*, not just the magnitudes. The contract reviewer beats the naive one entirely on false positives (0.05 vs 0.25, then 0.10 vs 0.55) — same direction, both corpora. The over-flagger has perfect recall but maximal FP: recall alone would crown it, the pair sinks it. The coverage-gap reviewer has near-perfect FP but low recall: FP alone would rank it best, the pair sinks it. That two different degenerates are caught by two *different* halves of the pair is the mechanism working — a single-axis fluke cannot produce it. And recall sits at 1.00 for every real reviewer on both corpora: the saturation that defines the limitation, reproduced.



Why a closed-rule gate cannot police a process whose novelty is the point.

Part 6 — Does it work? The honest verdict

Stated plainly, scoped exactly to what five experimental attempts support:

Demonstrated, and replicated across two independent corpora with real model calls: the paired recall / false-positive metric separates a contract-bound reviewer from a naive one, and separates both from a coverage-gap degenerate and an over-flagging degenerate. Neither number alone does this — the over-flagger is caught only by false positives, the blinded reviewer only by recall. The pairing is the mechanism, and it held.

Undemonstrated, with a known reason: discrimination on the recall axis between *plausible* reviewers. Recall saturates on textual cases, reproducibly. This is a property of how models read prose, and I report it as a limitation rather than patch over it.

Untested, and possibly hard to test: detection of the confident-misattribution reviewer — the most dangerous real-world failure — because that reviewer resisted three attempts at construction.

And the meta-result, which is the strongest thing here: across five hardened attempts the automated gate failed to catch a serious defect every time, and independent human audit caught every one. The repeated, heterogeneous failures point to a structural limitation of closed-rule gates: they can enforce known criteria well, but they are weak at detecting *novel* defects in the evidence-production process — because a gate can only check the failure modes its author anticipated, while the whole point of running such a process is that its output is not known in advance. The gate is complete only against the past; the process generates the future. (The companion meta-paper makes the stronger, mechanism-level version of this argument and defends it against the "your gate was just misconfigured" objection.) Either way, this is a recursive demonstration of CAA's

founding rule — the producer of evidence cannot be the one who assures it — produced by the very attempt to test something else.

So: **does it work?** It works where I could test it, it honestly maps where it does not, and in failing to police itself automatically it proved its own central premise. For a framework whose entire pitch is "don't trust the reviewer, verify it," there is no more fitting result.

Part 7 — Threats to validity

A report that argues "don't trust without calibrating" must be honest about its own limits. These are the real ones, not performative ones.

The cases are textual, not real systems. This is the most important threat. The experiment measures whether models detect risks *described in prose*, not whether they review production systems, codebases, telemetry, or live design artifacts. The recall saturation is direct evidence of the gap: a model reading a description with the cause embedded *finds it* far more easily than it would diagnose a running system. Everything demonstrated here is demonstrated for controlled textual incident pairs; generalization to live artifacts is a hypothesis, not a result.

Corpus size and authorship. n = 20 matched pairs per corpus — directional, not powered for strong statistics. I wrote the cases and assigned their ground-truth causes, so the corpus inherits my judgment; a second independent annotator would strengthen it. The effect direction is consistent and replicated across two corpora, but the magnitudes should be read as observations, not population estimates.

Unintended cues. A failure case could leak its answer through wording rather than mechanism. I mitigated this with a "blind reader" test (no sentence may state the conclusion; no taxonomy label in the text) applied to every case — and it took three full rewrites to pass it, with the failures documented in the saga. Residual subtle cues cannot be ruled out entirely.

Single model family. One reviewer model and one (distinct) judge model. The two-corpus replication establishes robustness *to the corpus*, not *to the model*. Whether the findings hold across model families is untested.

Language. The whole experiment — cases, reviewer prompts, judge, and outputs — ran in Brazilian Portuguese; this report presents it in English. Generalization to other languages is untested, and the recall-saturation result especially could differ in another language. The raw records in the repository are kept in their original Portuguese rather than translated, so the artifact reflects exactly what was run.

Constructed controls. The degenerate reviewers are degenerate *by construction*. The study therefore demonstrates discrimination of *known structural* degeneracy (coverage gap, over-flagging), not detection of a subtly bad reviewer that arises naturally — which is exactly the capability the misattribution control failed to test.

The judge is itself a model. Category-match scoring uses an LLM judge, which can err. Mitigated by using a judge model distinct from the reviewer and by independent raw-data reconstruction of every number, but not eliminated.

Naming these does not weaken the contribution; it scopes it. The demonstrated results survive every threat above; the undemonstrated ones are undemonstrated *because* of threats I am stating rather than hiding.

Part 8 — Is it worth it? When to use CAA, and when not

Honesty about fit is part of the value. CAA is not free — it adds an independent assurance plane, which is real overhead.

Use it when AI participates in reviewing or operating systems where being *confidently wrong* is expensive: regulated environments, financial systems, anything where an unaudited bad call carries real cost. The more you rely on AI judgment, the more you need to know that judgment is calibrated.

Do not use it when the stakes do not justify the overhead. A formatting helper, a low-risk internal tool, a throwaway script — these do not need an assurance plane, and bolting one on is waste. The risk-class triage exists precisely so the heavy machinery only runs where it earns its keep.

The minimum adoption path — because no one buys the whole architecture at once: start by plugging *one* reviewer under contract into a process you already have. Make it emit a structured verdict with traceable evidence instead of prose. Measure it against a handful of your own real incidents. That single step — one calibrated instrument, in one place — delivers most of the value and proves the idea on your own ground before you invest further.

Part 9 — Why this matters, and what it is worth

The deeper value of CAA is not the pipeline or the ten reviewers. It is the **contract** — the verdict schema, the origin rule, the paired metric. Everything else is a substitutable implementation behind that interface. Lay the stable interface, evolve the engines later. This is what makes CAA a *plumbing* layer that survives the churn of which model or which framework is fashionable this quarter.

And the timing matters. As AI moves from writing code to reviewing it to operating systems, the question "who assures the AI?" stops being academic and becomes an operational and regulatory necessity. Right now the honest answer in most organizations is silence or vendor trust. CAA is one answer with the receipts to back its limits — which, in a field full of unfalsifiable promises, is itself the differentiator.

I did not set out to prove CAA works. I set out to break it, documented every place it broke, and what survived is stated without inflation. That is the posture I think this field needs, and it is the posture this document is written in.

Appendix — The companion papers and artifacts

This report is the readable overview. The rigorous detail lives in three genre-separated papers and the reproducibility artifacts, all in this repository:

- **Position paper** (`docs/papers/position-paper.md`) — the framework, formally, positioned against the assurance-case literature.
- **Empirical study** (`docs/papers/empirical-study.md`) — the pre-registered falsification, with full method, metrics, and the two-corpus replication.
- **Meta-paper** (`docs/papers/meta-paper.md`) — the recursive gate-failure result, with the threats-to-validity analysis that earns the structural claim.
- **Verdict contract** (`spec/verdict-contract.schema.json`) — the pluggable interface.

- **Experiment** (`experiments/`) — the protocol and runnable harness; every numeric claim in this report is reconstructable from the raw per-call records, independently of the harness, with run provenance verified as real.

All results stated as of the five attempts completed for v0.1. This is a pilot program, honestly scoped, not a finished product — which is exactly what the framework would require me to say.

— Rudson Carvalho, Independent Researcher